

A Practical Guide to Gradient descent

TOPIC -- GRADIENT DESCENT

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

-- 01 --

Because each step is taken in the steepest direction, steepest-descent steps alternate between directions aligned with the extreme axes of the elongated level sets. When $\kappa(\mathbf{A})$ is large, this produces a characteristic zig-zag path. The poor conditioning of $\mathbf{A}$ is the primary cause of the slow convergence, and orthogonality of successive residuals reinforces this alternation. As shown in the image on the right, steepest descent converges slowly due to the high condition number of $\mathbf{A}$, and the orthogonality of residuals forces each new direction to undo the overshoot from the previous step. The result is a path that zigzags toward the solution. This inefficiency is one reason conjugate gradient or preconditioning methods are preferred.

-- 02 --

Modifications Gradient descent can converge to a local minimum and slow down in a neighborhood of a saddle point. Even for unconstrained quadratic minimization, gradient descent develops a zig-zag pattern of subsequent iterates as iterations progress, resulting in slow convergence. Multiple modifications of gradient descent have been proposed to address these deficiencies.

-- 03 --

Gradient descent is based on the observation that if the multi-variable function $f(\mathbf{x})$ is defined and differentiable in a neighborhood of a point $\mathbf{a}$, then $f(\mathbf{x})$ decreases fastest if one goes from $\mathbf{a}$ in the direction of the negative gradient of f at $\mathbf{a}$, i.e., $-\nabla f(\mathbf{a})$. It follows that, if

-- 04 --

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x} - 2 \mathbf{x}^T \mathbf{b}, \quad \{\displaystyle f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x} - 2 \mathbf{x}^T \mathbf{b},\}$$

-- 05 --

$$\nabla f(\mathbf{x}) = 2 \mathbf{A} \mathbf{x} - 2 \mathbf{b}. \quad \{\displaystyle \nabla f(\mathbf{x}) = 2 \mathbf{A} \mathbf{x} - 2 \mathbf{b}.\}$$

-- 06 --

$$f(\mathbf{x}(1)) = 0.5((-2.48)^2 + (-1.00)^2 + (6.28)^2) = 23.306. \quad \{\displaystyle f(\mathbf{x}(1)) = 0.5((-2.48)^2 + (-1.00)^2 + (6.28)^2) = 23.306.\}$$

-- 07 --

Momentum or heavy ball method Trying to break the zig-zag pattern of gradient descent, the momentum or heavy ball method uses a momentum term in analogy to a heavy ball sliding on the surface of values of the function being minimized, or to mass movement in Newtonian dynamics through a viscous medium in a conservative force field. Gradient descent with momentum remembers the solution update at each iteration, and determines the next update as a linear combination of the gradient and the previous update. For unconstrained quadratic minimization, a theoretical convergence rate bound of the heavy ball method is asymptotically the same as that for the optimal conjugate gradient method. This technique is used in stochastic gradient descent and as an extension to the backpropagation algorithms used to train artificial neural networks. In the direction of updating, stochastic gradient descent adds a stochastic property. The weights can be used to calculate the derivatives.

-- 08 --

Choosing the step size and descent direction Since using a step size η that is too small would slow convergence, and a η too large would lead to overshoot and divergence, finding a good setting of η is an important practical problem. Philip Wolfe also advocated using "clever choices of the [descent] direction" in practice. While using a direction that deviates from the steepest descent direction may seem counter-intuitive, the idea is that the smaller slope may be compensated for by being sustained over a much longer distance. To reason about this mathematically, consider a direction $\mathbf{p}_n$ and step size η_n and consider the more general update:

-- 09 --

so the sequence $(\mathbf{x}_n)$ converges to the desired local minimum. Note that the value of the step size η is allowed to change at every iteration. It is possible to guarantee the convergence to a local minimum under certain assumptions on the function f (for example, f convex and ∇f Lipschitz) and particular choices of η . Those include the sequence

-- 10 --

Solution of a non-linear system Gradient descent can also be used to solve a system of nonlinear equations. Below is an example that shows how to use the gradient descent to solve for three unknown variables, x_1 , x_2 , and x_3 . This example shows one iteration of the gradient descent. Consider the nonlinear system of equations

-- 11 --

Comments Gradient descent works in spaces of any number of dimensions, even in infinite-dimensional ones. In the latter case, the search space is typically a function space, and one calculates the Fréchet derivative of the functional to be minimized to determine the descent direction. That gradient descent works in any number of dimensions (finite number at least) can be seen as a consequence of the Cauchy–Schwarz inequality, i.e. the magnitude of the inner (dot) product of two vectors of any dimension is maximized when they are colinear. In the case of gradient descent, that would be when the vector of independent variable adjustments is proportional to the gradient vector of partial derivatives. The gradient descent can take many iterations to compute a local minimum with a required accuracy, if the curvature in different directions is very different for the given function. For such functions, preconditioning, which changes the geometry of the space to shape the function level sets like concentric circles, cures the slow convergence. Constructing and applying preconditioning can be computationally expensive, however. The gradient descent can be modified via momentums (Nesterov, Polyak, and Frank–Wolfe) and heavy-ball parameters (exponential moving averages and positive-negative momentum). The main examples of such optimizers are Adam, DiffGrad, Yogi, AdaBelief, etc. Methods based on Newton's method and inversion of the Hessian using conjugate gradient techniques can be better alternatives. Generally, such methods converge in fewer iterations, but the cost of each iteration is higher. An example is the BFGS method which consists in calculating on every step a matrix by which the gradient vector is multiplied to go into a "better" direction, combined with a more sophisticated line search algorithm, to find the "best" value of η . For extremely large problems,

where the computer-memory issues dominate, a limited-memory method such as L-BFGS should be used instead of BFGS or the steepest descent. While it is sometimes possible to substitute gradient descent for a local search algorithm, gradient descent is not in the same family: although it is an iterative method for local optimization, it relies on an objective function's gradient rather than an explicit exploration of a solution space. Gradient descent can be viewed as applying Euler's method for solving ordinary differential equations $x'(t) = -\nabla f(x(t))$ to a gradient flow. In turn, this equation may be derived as an optimal controller for the control system $x'(t) = u(t)$ with $u(t)$ given in feedback form $u(t) = -\nabla f(x(t))$.